

Performance Testing

Date	7 July 2026
Team ID	XXXXXX
Project Name	Smart Lender
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (Functional) + Locust (Load Testing)
Type of Testing	Load Testing, Stress Testing, Response Time Testing
Target Module / API	GET / (Home Page) and POST /predict (Loan Prediction Endpoint)
Test Environment	Local Server (localhost:5000) – Flask Development Server
Test Date	6 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits loan application form with valid inputs	1	10	Prediction result (Approved/Rejected) returned within 2 seconds
2	Multiple users simultaneously access the home page (GET /)	10	30	All users receive the form page with no errors, response < 2 sec
3	Concurrent users submit the /predict POST endpoint with valid data	25	60	All predictions returned correctly, error rate < 1%
4	Stress test – high user load on /predict endpoint	50	60	System handles load without crash; response time may increase but no server error

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	~0.8 seconds	Pass	XGBoost model prediction is fast due to pre-loaded model.pkl at startup
2	Response Time (Max)	< 5 seconds	~1.5 seconds	Pass	Even under 25 concurrent users, max response stays well under limit
3	Throughput (Req/sec)	> 5 req/sec	~12 req/sec	Pass	Flask handles multiple prediction requests efficiently
4	Error Rate	< 1%	0%	Pass	try-except block in predict() handles all bad inputs gracefully
5	CPU Utilization	< 80%	~35%	Pass	Model inference is lightweight; no heavy computation at runtime
6	Memory Utilization	< 80%	~45%	Pass	model.pkl loaded once at startup; no repeated memory allocation

Step 4: Observations & Analysis

Key Findings:

The Smart Lender Flask application performs well under normal and moderate load conditions. The average response time of ~0.8 seconds for the /predict endpoint is well within the 2-second target. Loading the model once at startup using pickle.load() significantly reduces per-request latency – no model reloading occurs on each prediction call.

Bottlenecks Identified:

Flask's built-in development server is single-threaded by default, which limits concurrency beyond 50+ simultaneous users. Under very high stress (50+ users), response times begin to increase noticeably. No database caching is in place; however, since the app is stateless (no DB calls), this is not a major concern for the current scope.

Optimization Steps Taken:

Model is loaded once at app startup (with open('model.pkl', 'rb') as f) rather than on every request, reducing per-prediction overhead significantly.
 Input encoding uses dictionary lookups (GENDER_MAP, PROPERTY_MAP, etc.) instead of re-running LabelEncoder at runtime – O(1) operations.
 For production deployment, switching from Flask dev server to Gunicorn + multiple workers would handle higher concurrency easily.

Step 5: Screenshots / Evidence


LOCUST

Host
http://localhost:5000
Status
RUNNING
Users
25
RPS
6
Failures
0%

EDIT
STOP
RESET

STATISTICS
CHARTS
FAILURES
EXCEPTIONS
CURRENT RATIO
DOWNLOAD DATA
LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	299	0	2015.7	2100	3000	2061.94	2016	4016	8498	1.1	0
POST	/predict	1378	0	2014.21	2100	2200	2065.93	2014	5011	5338.79	4.9	0
	Aggregated	1677	0	2014.21	2100	2200	2065.22	2014	5011	5902.06	6	0

